Debugging: A systematic review on the integration of generative AI and gamification in programming education

1st Thuany Guimarães Pereira
Informatics Center - CIn
Federal University of Pernambuco
Recife, Brazil

0009-0007-4822-3800
2nd Douglas Araújo Silva
Informatics Center - CIn
Federal University of Pernambuco
Recife, Brazil
0009-0006-1448-3164

3rd Nathanael Noberto da Silva
Informatics Center - CIn
Federal University of Pernambuco
Recife, Brazil
0000-0002-8153-0569

4st André Luiz Bartholomeu Ribeiro
Informatics Center - CIn
Federal University of Pernambuco
Recife, Brazil
0009-0002-7809-1160

5nd Luiz Guylherme Avelino Rodrigues
Informatics Center - CIn
Federal University of Pernambuco
Recife, Brazil
0009-0002-5328-6949

Abstract*— This work presents a Systematic Literature Review (SLR) focused on mapping the state of the art of teaching and supporting debugging in beginner software programming classes (CS1). The research investigates mediation through gamification and Generative Artificial Intelligence (GAI). Ten primary studies with high methodological rigor were analyzed, revealing a methodological rupture where LLMs act as semantic error translators. The results point to a paradox: while GAI drastically reduces frustration and cognitive load, it generates risks of an "illusion of competence" and algorithmic dependency. It is concluded that the effectiveness of these technologies depends on a Socratic and gamified instructional design.

Keywords*— Debugging, Gamification, Generative AI, Beginner Programmers, RSL.

I. Introduction

When thinking about the nuances of contemporary computing education, the importance of building a consistent knowledge base for beginner students in the field emerges as a necessary and immediate reflection to ensure the quality level during the later stages of the learning process and future professional performance of students. The development of these skills, especially regarding introductory programming courses, requires students to master complex concepts, which in turn demand appropriate formative feedback to maintain student motivation and retention in their respective courses, as per [1].

Following the reflections of [2], we realize that when immersed in the practical moment of the debugging process, a beginner programming student is expected to encounter error messages, and in the absence of pedagogical support capable of making them understand the meaning of that message, this practice becomes a frustrating experience that causes cognitive overload for that student. It is at this point that we can see that the lack of support for understanding errors induces students into a cycle of trial and error just to satisfy the compiler, without understanding the cause of that message.

Educators daily seek innovative approaches focused on mitigating the frustration and cognitive overload of debugging in CS1. It is noted that the literature has mapped the use of educational games to support the teaching and learning of programming, as well as the state of the art on gamification in STEM courses, showing an extremely positive impact on the engagement, motivation, and attitude of programming students [3]. Gamification has proven to be an interesting approach for this context, as the application of its elements in programming education increases motivation and improves the understanding of content, according to [4], where in practice, motivation has an increase reflected by the understanding of software tests. The use of serious games is one of the practical ways to apply gamification, as they consist of the use of technologies and digital game environments applied to an educational context, promoting student-centered learning through interactivity and immediate feedback, using their immersive nature as a drive for the student to want to investigate and experiment with errors [5]. Empirical studies demonstrate that the use of these games reduces dropout intention, in addition to significantly improving the self-perception and

confidence of these learners when applying debugging skills [5].

Despite the noticeable benefits of gamification on learner resilience in the long term, metacognitive obstacles for solving programming problems limit the advancement of beginner programmers. Because they do not perceive the root of these difficulties, students consider the use of Generative Artificial Intelligence (GAI), such as Large Language Model (LLM) tools, as a shortcut to obtain support for error resolution and to bypass situations, mainly due to the immediate availability of feedback [6]. However, its use for the integral construction of code to solve demands brings cognitive harms and deficits in the next stages of learning and subsequent professional performance, making it necessary to consider that this form of GAI support, as studies such as [7] indicate, amplifies these cognitive difficulties, bringing a dangerous "illusion of competence" to the beginner. Finally, GAI fails to offer support aimed at thinking about eliminating the need for knowledge about the multiple stages of the debugging process to mitigate code problems, since it requires the beginner to have this skill to be able to understand and fix the code generated by the machine itself, as demonstrated by [8]. Without proper knowledge, the machine-generated code transfers the cognitive load instead of eliminating it.

Although there are works focused on the isolated use of gamification and GAI in programming education, there is a certain lack of mapping on how their combination and structuring can be performed with a focus on promoting the concrete learning of the skills necessary for the debugging process for beginners, but in a healthy way, without generating technological dependence, globally empowering the learner's independence.

With this work, we intend to map the state of the art of teaching and supporting debugging in beginner software programming classes, using mediation through gamification and GAI, performing a Systematic Literature Review (SLR) to identify trends, methodological challenges, and direct future work in the field of Informatics in Education.

II. RELATED WORK AND THEORETICAL FOUNDATION: THE CURRENT SCENARIO OF DEBUGGING EDUCATION

This section presents the current overview of the literature regarding the challenges of teaching code debugging and the technological approaches used to mitigate them. In this path, Subsection II.A establishes the cognitive and emotional barriers faced by beginner programmers. Subsection II.B discusses the impacts of seeking unsupervised help in GAI tools. Next, Subsection II.C addresses how the scenario of Socratic hints and gamified environments has been reported in the literature, concluding in II.D where the scientific gap related to the hybrid integration of these technologies is explained.

A. The emotional and metacognitive labyrinth of debugging

Teaching debugging to beginner students is one of the greatest pedagogical challenges faced in computing. Literature reveals that due to a lack of metacognitive awareness, learning to program becomes a more difficult path, as this deficit impairs the elaboration of metacognitive strategies for understanding and executing activities, with metacognition being a crucial skill in learning programming [7]. When encountering an error message, without proper metacognitive awareness to logically analyze the error, students report that they "panic" and, in an attempt to solve the problem, begin a saga of blind trial and error seeking to get rid of the error presented in the message, to correct it even without an exact understanding of what it is [9].

This "panic" has consequences for the learning process, because the shock of the discrepancy between what the student expected the code to do and the actual error presented ends up generating a "cognitive impasse" (Confusion) [10]. With the difficulty in elaborating metacognitive strategies to logically analyze the situation, learners do not know how to get out of this impasse, which ends up turning confusion into severe frustration (logical stress), creating a sense of incapacity and destroying the learner's confidence, which is one of the greatest causes of dropout from their respective courses in their initial phase [11].

B. Seeking Help and the "Illusion of Competence" generated by GAI

As an alternative to escape from stress and incapacitating frustration, learner behavior frequently tends to seek executive help and quick-shortcut solutions, instead of deepening their knowledge and finding deep solutions with greater impact on their learning [12]. It is in this context that GAI tools have become immediate relief for students, proving useful due to the speed of the response. The problem, however, lies in the way they are used: due to a lack of instruction, learners outsource debugging to the machine without previously trying to solve it independently or validating if the GAI responses are suitable for solving the problem [12] [13].

Recent literature has been warning about the inherent

dangers of this practice, reporting that the free use of GAI worsens the resolution of cognitive impasses caused by the debugging process, instead of functioning as this immediate relief in an effective and healthy way. Delivering a ready-made code instead of proper explanation about the construction and functioning promotes cognitive dissonance in the student, where they begin to have a dangerous "illusion of competence," hindering the development of autonomous logical reasoning intrinsic to this type of activity [7].

C. Current Mediation Strategies: Socratic Hints and Gamification

One way to combat the outsourcing of reasoning to the machine is to change the way students interact with GAI tools. According to recent research, for this change in behavior, it is necessary for the design of educational tools to "tame" LLMs so they act only as tutors, providing contextualized feedback and structured hints, which automatically induces the student to maintain conceptual focus on what is required by the debugging process and the consequent resolution of errors with fewer failed attempts [14] [15].

At the same time that instructional innovations using GAI are recommended to solve this problem, works talking about gamification have been recognizing and validating this mechanism as promising in programming education, proving effective in the emotional barrier of debugging and serving to improve student participation and positively impact learning [3]. Simultaneously, the immersive and engaging nature of games encourages students to higher levels of investigation, experimentation, and reuse [5], as well as promotes the use of scaffolding and testing to support programming education [4] even before the popularization of GAI.

D. The Gap of Hybrid Integration in Programming Education

Taking into account the possibility of generating dependence on the use of GAI, it is necessary to point out that this technology also has the capacity to offer vital hints. Researchers have been advocating for a profound pedagogical change to align its use in a psychologically healthy way, with the creation of integrating methods that can actively use GAI in gamified environments (Serious Games) focusing on ensuring the development of critical knowledge from practical activities—that is, teaching the learner to think while performing activities, setting aside the fierce war over the detection of GAI use [16] [17].

The literature is still deficient regarding the combination of teaching debugging to beginner students with the hybrid integration of gamification and GAI. Aiming to fill this gap, this article proposes to map how joint interactive approaches are being designed, seeking to understand the impacts on the autonomy and engagement of beginner programming students.

[Faltou a Seção de Metodologia]

III. RESULTS

A. Metadata and Publication Demographics

Systematic extraction validated 10 primary studies that address the process of debugging mediated by artificial intelligence (AI) and gamification, presenting high methodological rigor (average quality score of 4.85). The time distribution, illustrated in FIG.1 (Year of publication), shows an absolute concentration in the last two years, with 70% of the evidence dated 2025 and 30% 2024

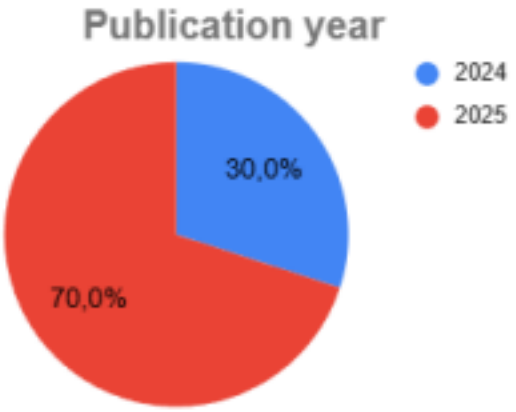
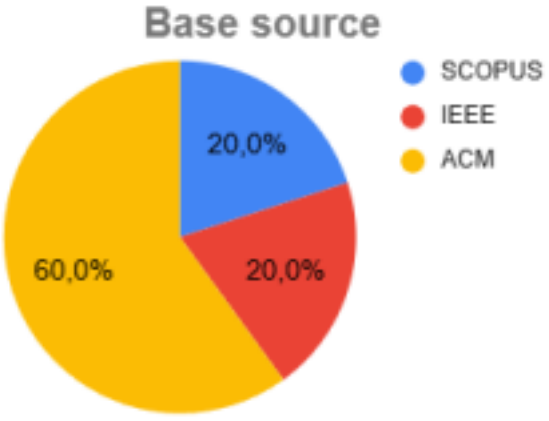
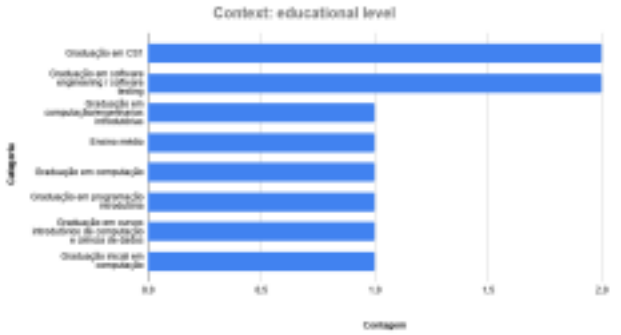


FIG. 2 (base source) indicates the predominance of the ACM digital library (60%), followed by IEEE Xplore and SCOPUS (20% each). The consolidated geographical analysis in FIG.3 highlights the leadership of the United States (3 studies) and India (2 studies), with unique contributions from Germany, Brazil, Australia, China, and Singapore.

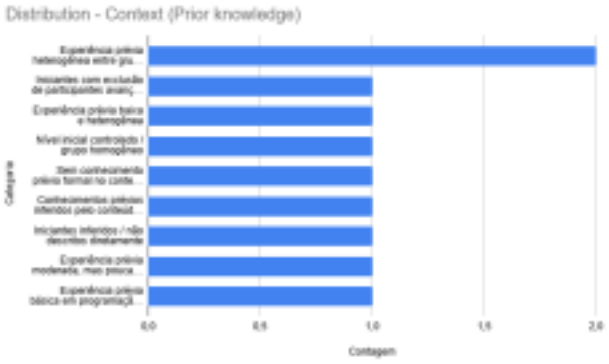


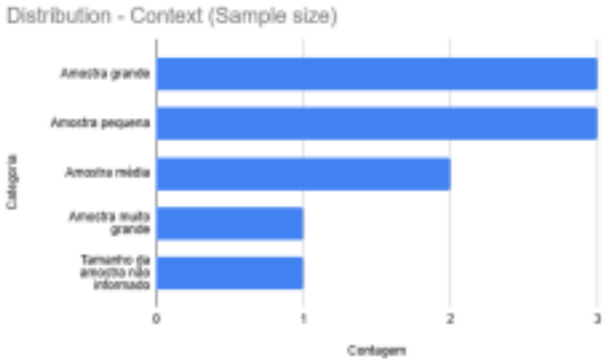
B. Educational and demographic context

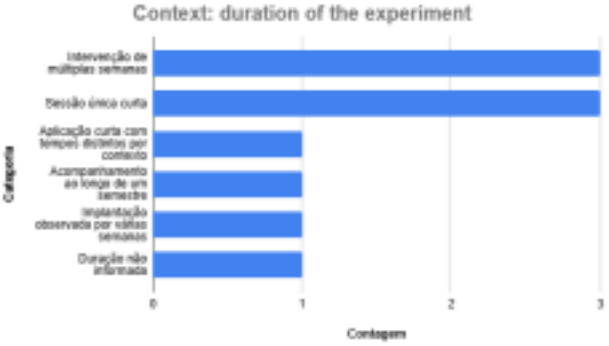
The population data extracted attest to a focus on initial higher education. FIG. 4 (Context: educational level) reveals applications concentrated in higher education, with primary incidences in subjects of "Bachelor's degree in CS1" (2 studies) [23, 26] and "Bachelor's degree in software engineering/software testing" (2 studies) [27, 33], in addition to generic introductory programming disciplines [31, 32].



OOs dados populacionais extraídos atestam um foco no ensino superior inicial. FIG. 4 (Contexto: nível educacional) revela aplicações concentradas no ensino superior, com incidências primárias em disciplinas de "Bacharelado em CS1" (2 estudos) [23, 26] e "Bacharelado em engenharia/testes de software" (2 estudos) [27, 33], além das disciplinas genéricas de programação introdutória [31, 32].

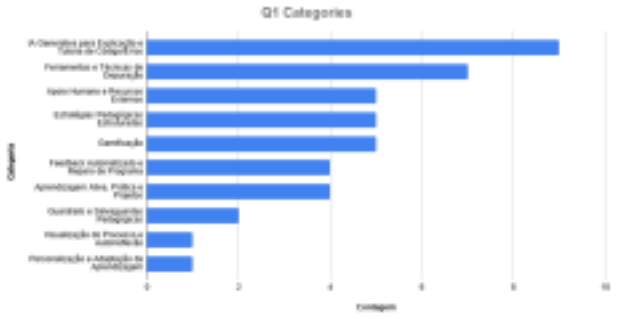






C. Pedagogical Technologies and Strategies (Q1)

Fig. 8 (Categories Q1) shows the technical approaches used to cure the problem of a learner with software failures. The category "AI Generative for Explanation and Tutoring of Code/Errors" prevails in isolation, conducting 9 of 10 primary studies [22, 23, 26, 27, 30, 31, 32]. "Automated Feedback and Program Repair" tools inteTo neutralize passive automation, synthetic interventions were linked to "Structured Pedagogical Strategies" (6 occurrences) [27, 31] and "Gamification" mechanics (5 occurrences) [17]. Additionally, the graph points out the use of "Active Learning, Practice and Projects" methodologies in 4 works [28, 32], fundamental to contextualize the process of debugging failures.



D. Educational Benefits and Problem Solving (QS2 and QS4)

Quantification of positive outcomes is detailed in FIGS.9 and 10 ("QS2 Categories" and "QS4 Categories"). AI acted as a stress mitigator, reflected in the "Reduction of Frustration, Anxiety and Cognitive Load", which

recorded the maximum score of 10 empirical incidences [26, 32]. The use of tools did not limit the investigative capacity, fostering "Logical Reasoning, Critical Thinking and Conceptual Understanding" (10 occurrences) [30, 31].

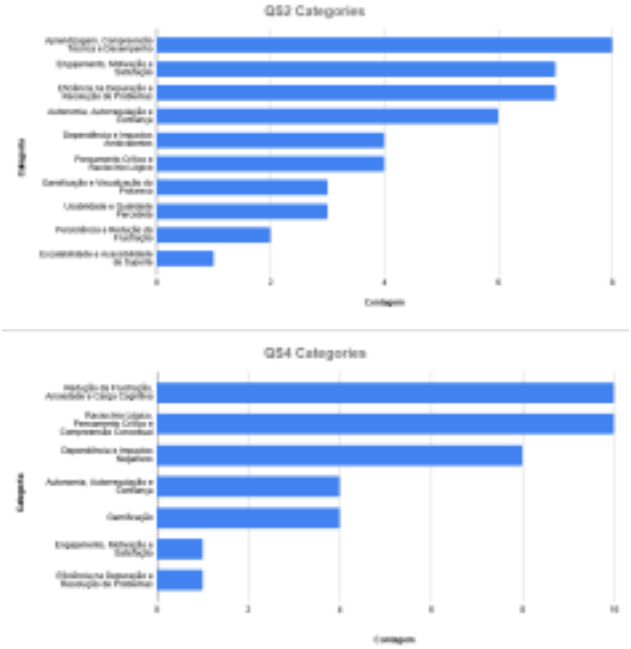
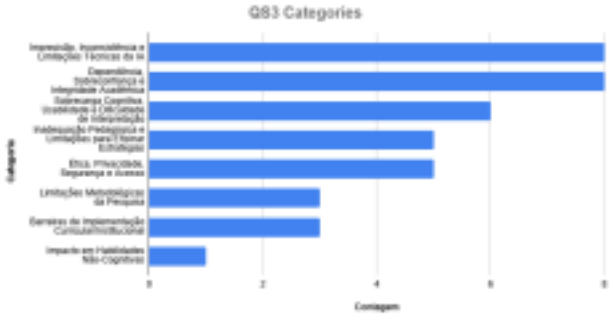


Fig. 9 (QS2) reinforces these gains, indicating direct impacts on "Learning, Technical Understanding and Performance" (8 mentions). The synergy between dialogical explanations and gamification raised the "Efficiency in Debugging and Problem Solving" (7 occurrences) and the "Engagement, Motivation and Satisfaction" (7 occurrences). As a behavioral result, the consolidation of "Autonomy, Self-regulation and Trust" was observed in 6 evaluations.

E. Challenges, Limitations and Risks (QS3)

Fig. 11 (QS3 Categories) isolates the critical obstacles inherent in implementing AI in the educational process. The literature reports a severe "Dependency, Overconfidence and Academic Integrity" (8 occurrences), highlighting the risk of learners using AI agents as shortcuts to automatic writing of code.



Architectural anomalies manifested through the "Imprecision, Inconsistency and Technical Limitations of AI" (8 mentions), indicating the occurrence of algorithmic hallucinations. Beginner students also faced "Cognitive Overload, Usability and Difficulty of Interpretation" (6 mentions) when operating complex prompts. Issues of "Ethics, Privacy, Security and Access" were identified in 6 studies, accompanied by "Pedagogical Inadequacy and Limitations for Teaching Strategies" (5 occurrences), where the tool failed to instruct the correct debugging methodology without human intervention.

IV. Discussion

A. *The role of generative AI in introductory teaching*

The demographics outlined in the context charts (Educational level and Prior knowledge) reaffirm a historical diagnosis in computer science education: CS1 students and engineers face a severe entry barrier due to the opacity of traditional compiler error messages [23, 26, 32]. The massive prevalence of "Generative AI for Explanation and Tutorship" (9 occurrences in Fig. Q1) demonstrates a methodological breakdown [22, 27, 31]. Large language model (LLMs) tutors assume the role of semantic translators, converting logical and syntax flaws into comprehensible dialogues. The strong occurrence of "Structured Pedagogical Strategies" and "Gamification" in the same Fig. (Q1) highlights that mere access to technology is insufficient; the effectiveness of AI depends on an instructional design that integrates technology into serious game logics and project-based learning [28, 33].

B. *Cognitive Relief Paradox and Algorithmic Dependence*

The contrast between positive impact data (QS2 and QS4) and detected risks (QS3) establishes the main paradox of this theme. On the one hand, synthetic tools eliminated the primary vector of university evasion in these disciplines, reflected in the 10 incidences of "Reduction of Frustration and Cognitive Load" and 7 mentions to increase "Engagement" [26, 28, 32]. The simultaneous gain in "Logical Reasoning and Critical Thinking" (10 occurrences) corroborates the premise that AI can act as a cognitive facilitator (scaffolding), which unlocks technical impasses and allows the student to focus on the structural logic of the algorithm [30, 31].

However, the metrics of Fig. QS3 demonstrate that this relief quickly degenerates into a technological crutch. The empirical rate of "Dependence and Overconfidence" (8 occurrences) indicates that, lacking friction, students inhibit investigative capacity and trust blindly in the automated solution [22, 23, 127, 30]. AI fixes the code, but the debugging skills are outsourced, nullifying the programmer’s future autonomy.

C. Pedagogical implications and technical limitations

The "Pedagogical Inadequacies" (5 occurrences) and the flaws for "Inaccuracy and Technical Limitations" (8 occurrences) reported in Fig.QS3 reveal that current LLMs lack epistemological verification rigor [6, 10, 15]. Generative AI prioritizes textual fluency over technical accuracy, often resulting in false positives in finding defects or encouraging misguided logical fixes [22, 27].

This technical instability, combined with "Cognitive Overload and Difficulty of Interpretation" (6 occurrences), requires academic institutions to implement restriction protocols (guardrails). The literature shows that sustainability of AI insertion in beginners' classes requires rigorous orchestration of the level of help provided by the system [27, 31]. The provision of complete resolutions (ready source code) must be systematically blocked in favor of Socratic methodologies, where the gamified bot leads the student to identify the fault itself through targeted questioning [31, 33].

References

[1] U. Z. Ahmed, S. Sahai, B. Leong, and A. Karkare, "Feasibility Study of Augmenting Teaching Assistants with Al for CS1 Programming Feedback," in SIGCSE TS 2025.

[2] D. J. Bouvier et al., "Teaching Programming Error Message Understanding," in CompEd 2023.

[3] M. Venter, "Gamification in STEM programming courses: State of the art," in EDUCON 2020.

[4] A. Almeida, D. D. S. Guerrero, and W. L. Andrade, "Fostering Problem-Solving in Introductory Programming Through Software Testing," in FIE 2025.

[5] C. Kazimoglu, "Enhancing Confidence in Using Computational Thinking Skills via Playing a Serious Game," IEEE Access, 2020.

[6] R. C. Sampaio, M. Sabbatini, and R. Limongi, "Diretrizes para o uso ético e responsável da Inteligência Artificial Generativa," Editora Intercom, 2024.

[7] J. Prather et al., "The Widening Gap: The Benefits and Harms of Generative Al for Novice Programmers," in ICER '24.

[8] S. Nguyen et al., "How Beginning Programmers and Code LLMs (Mis)read Each Other," in CHI '24.

[9] D. J. Bouvier et al., "Teaching Programming Error Message Understanding," in CompEd-WGR 2023.

[10] R. Batra and Z. Atiq, "Understanding Students' Frustration and Confusion during a Programming Task," in FIE 2023.

[11] K. Banweer and D. Trytten, "A Qualitative Study of How Computer Science Students Develop Debugging Skills," in FIE 2024.

[12] J. Penney et al., "Understanding Programming Students' Help-Seeking Preferences in the Era of Generative Al," in CompEd 2025.

[13] X. Long et al., "Understanding and Enhancing CS Students Interaction Experience with Al Coding Assistant Tools," ACM Trans. Softw. Eng. Methodol., 2025.

[14] R. Deoraj, J. V. S. G. Kurivella, and M. Moraes, "Guiding, Not Giving: Analyzing the Effectiveness of Guided Problem-Solving in CSI," in FIE 2025.

[15] C. Ayora et al., "Rethinking How to Teach Analysis and Modelling of Business Requirements: A Serious Game Integrating GenAI," in MODELS-C 2025.

[16] M. Triantafyllidou, H. Apostolidis, and L. Moussiades, "CODEWIZARDY, Online Application Supporting Instructors and Students to Learn Programming," in EDUCON 2024.

[17] D. B. Silva, D. R. Carvalho, e C. N. Silla Jr., "A Clustering-Based Computational Model to Group Students With Similar Programming Skills," IEEE Transactions on Learning Technologies, 2024.

[18] Y. Yang, A. Mei, e C. Yang, "An Empirical Study of MBFL on Novice Programs Across Different Programming Languages," Int. Journal of Software Engineering and Knowledge Engineering, 2025.

[19] X. Xiao et al., "An assessment framework of higher-order thinking skills based on fine-tuned large language models," Expert Systems With Applications, 2025.

[20] W. Jang et al., "Analyzing Communication Logs in Pair Programming: A Comparison of Human- and LLM-Based Approaches," ETRA 2024.

[21] H. M. Jamil e X. Mou, "Automated Feedback and Authentic Assessment for Online Computational Thinking Tutoring Systems," in FIE 2017/2018.

[22] K. H. Levin et al., "ChatDBG: Augmenting Debugging with Large Language Models," Proc. ACM on Software Engineering (FSE), 2025.

[23] L. N. Gomes, C. F. Fraga, M. Holanda, e D. Da Silva, "ChatGPT as a Teaching Assistant in a CS1 course: An Experience Report," 2024/2025.

[24] S. Kannam et al., "Code Interviews: Design and Evaluation of a More Authentic Assessment for Introductory Programming Assignments," in SIGCSE 2025.

[25] Y. Zhou et al., "CodeEduBench: A Multidimensional

Benchmark for Evaluating Al-Driven Code Generation Models," in ICAIES 2025.

[26] S. Yang et al., "Debugging with an Al Tutor: Investigating Novice Help-seeking Behaviors and Perceived Learning," in ICER '24.

[27] O. Kurniawan et al., "Designing for Novice Debuggers: A Pilot Study on an Al-Assisted Debugging Tool," in Koli Calling '25, 2025.

[28] S. V. P. Masetty e S. Vallamulla, "Enhancing Novice Programming Education through Al-Driven Mentorship and Project-Based Learning," VedaVerse, 2024.

[29] H.-W. Chao e A. Y. Chang, "Exploring the Role of Al in Transforming Programming Education at Universities," 2025.

[30] M. Sivasakthi e A. Meenakshi, "Generative Al in Programming Education: Evaluating ChatGPT's Effect on Computational Thinking," SN Computer Science, 2025.

[31] X. Gong, Z. Li, e A. Qiao, "Impact of generative Al dialogic feedback on different stages of programming problem solving," Education and Information Technologies, 2025.

[32] B. Adhikari, S. Dhakal, e A. Jha, "Maximizing Student Engagement in Coding Education with Explanatory Al," 2024.

[33] P. Straubinger, T. Greller, e G. Fraser, "Sojourner under Sabotage: A Serious Testing and Debugging Game," in FSE Companion '25, 2025.